



Building a Blazor WebAssembly Client

Step-by-Step Guide — ASP.NET Core Web API with JWT Authentication

Author: Training Reference | **Framework:** .NET 10 | **Project:** EmployeeApp



Table of Contents

1. Solution Overview
2. Step 1: Configure HttpClient
3. Step 2: Update _Imports.razor
4. Step 3: Create Login Page
5. Step 4: Create Register Page
6. Step 5: Store JWT Token after Login
7. Step 6: Create CustomAuthStateProvider
8. Step 7: Register Services in Program.cs
9. Step 8: Update App.razor
10. Step 9: Update MainLayout.razor
11. Step 10: Send Token with API Requests
12. Step 11: Role-Based Employee List

Solution Overview

This guide explains how to build a **Blazor WebAssembly** client that connects to a secured ASP.NET Core Web API using **JWT (JSON Web Token)** authentication.

Project	Type	Purpose
EmployeeApp.Api	ASP.NET Core Web API	REST API with JWT auth, running on https://localhost:7021
EmployeeApp.Shared	Class Library	Shared DTOs used by both API and Client
EmployeeClient	Blazor WebAssembly	Browser-based client app, running on https://localhost:7280

API Endpoint Security Summary

Endpoint	Method	Who Can Access
/api/auth/register	POST	Anonymous
/api/auth/login	POST	Anonymous
/api/employees	GET	User + Admin
/api/employees/{id}	GET	User + Admin
/api/employees	POST	Admin only

/api/employees/{id}	PUT	User + Admin
/api/employees/{id}	DELETE	Admin only

Authentication Flow

How JWT Authentication Works in this App:

1. User submits email + password on Login page
2. Client POSTs to /api/auth/login
3. API verifies credentials and returns a JWT token
4. Client saves token to **browser localStorage**
5. Every subsequent API call includes the token in the Authorization: Bearer <token> header
6. API verifies the token and grants or denies access

① Step 1: Configure HttpClient in Program.cs

Purpose: Tell Blazor's HttpClient where the API lives. Every API call made from the client will use this base URL.

File: EmployeeClient/Program.cs

```
builder.Services.AddScoped(sp => new HttpClient
{
    BaseAddress = new Uri("https://localhost:7021/")
});
```

Line-by-Line Explanation:

- ▶ `builder.Services.AddScoped(...)` — Registers the `HttpClient` as a **Scoped** service in the DI (Dependency Injection) container. In Blazor WASM, Scoped means one instance per app lifetime.
- ▶ `new HttpClient { BaseAddress = new Uri("https://localhost:7021/") }` — Creates an `HttpClient` with the API's URL as its base. All relative URLs like `api/auth/login` will be resolved against this base.

💡 **Tip:** The trailing slash `/` at the end of the URL is **required**. Without it, relative paths like `api/auth/login` won't resolve correctly.

② Step 2: Update `_Imports.razor`

Purpose: Add global `@using` directives so all Razor components can use the shared DTOs and auth classes without repeating the using statements in every file.

File: `EmployeeClient/_Imports.razor`

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Web.Virtualization
@using Microsoft.AspNetCore.Components.WebAssembly.Http
@using Microsoft.JSInterop
@using EmployeeClient
@using EmployeeClient.Layout
@using EmployeeApp.Api.Models.Dtos    // <-- ADDED: LoginDto, RegisterDto, EmployeeDto
```

```
@using EmployeeApp.Api.Models           // <-- ADDED: AuthResponse
@using Microsoft.AspNetCore.Components.Authorization // <-- ADDED: AuthorizeView, CascadingAuthenticationState
@using EmployeeClient.Auth               // <-- ADDED: CustomAuthStateProvider
```

Line-by-Line Explanation (New Lines Only):

- ▶ `@using EmployeeApp.Api.Models.Dtos` — Imports the namespace containing **LoginDto**, **RegisterDto**, **EmployeeDto** from the Shared project.
 - ▶ `@using EmployeeApp.Api.Models` — Imports the namespace containing **AuthResponse** (the object returned after successful login, containing the JWT token).
 - ▶ `@using Microsoft.AspNetCore.Components.Authorization` — Imports Blazor's built-in authentication components: **AuthorizeView**, **CascadingAuthenticationState**, **AuthorizeRouteView**.
 - ▶ `@using EmployeeClient.Auth` — Imports our custom auth folder so **CustomAuthStateProvider** can be used in any Razor component.
-

③ Step 3: Create Login Page

Purpose: Build a form where the user enters their email and password. On submit, it calls the API's login endpoint.

File: EmployeeClient/Pages/Login.razor

```
@page "/login"
@inject HttpClient Http
@inject IJSRuntime JS
@inject CustomAuthStateProvider AuthStateProvider
@inject NavigationManager Nav
```

```

<h3 class="text-primary">Login</h3>

<div class="col-4">
    <EditForm Model="LoginRequest" OnValidSubmit="HandleLogin">
        <div class="mb-2">
            <label class="fw-bold">Email</label>
            <InputText class="form-control" @bind-Value="LoginRequest.Email"/>
            <ValidationMessage For="()"=>LoginRequest.Email"/>
        </div>
        <div class="mb-2">
            <label class="fw-bold">Password</label>
            <InputText type="password" class="form-control" @bind-Value="LoginRequest.Password"/>
            <ValidationMessage For="()"=>LoginRequest.Password"/>
        </div>
        <button class="btn btn-primary">Login</button>
    </EditForm>
</div>

@code {
    private LoginDto LoginRequest = new();

    private async Task HandleLogin()
    {
        var response = await Http.PostAsJsonAsync("api/auth/login", LoginRequest);
        if (response.IsSuccessStatusCode)
        {
            var authResponse = await response.Content.ReadFromJsonAsync<AuthResponse>();
            await JS.InvokeVoidAsync("localStorage.setItem", "token", authResponse!.AccessToken);
            AuthStateProvider.NotifyUserLoggedIn(authResponse.AccessToken);
            Nav.NavigateTo("/");
        }
        else
        {
            Console.WriteLine("Invalid Credentials");
        }
    }
}

```

Line-by-Line Explanation:

- ▶ `@page "/login"` — Registers this component as a routable page. Navigating to `/login` shows this component.
- ▶ `@inject HttpClient Http` — Injects the `HttpClient` registered in `Program.cs`. Used to call the API.
- ▶ `@inject IJSRuntime JS` — Injects the JavaScript interop service. Used to call browser JavaScript functions (like `localStorage`).
- ▶ `@inject CustomAuthProvider AuthServiceProvider` — Injects our custom auth provider to notify Blazor when login happens.
- ▶ `@inject NavigationManager Nav` — Injects the navigation service. Used to redirect after login.
- ▶ `<EditForm Model="LoginRequest" OnValidSubmit="HandleLogin">` — Blazor's form component. `Model` is the object being edited. `OnValidSubmit` calls `HandleLogin` only if validation passes.
- ▶ `<InputText @bind-Value="LoginRequest.Email"/>` — Two-way binding: user types in the box → value is stored in `LoginRequest.Email`. **Always use capital V** in `@bind-Value`.
- ▶ `<ValidationMessage For="()=>LoginRequest.Email"/>` — Displays validation error messages for the Email field when form is submitted.
- ▶ `private LoginDto LoginRequest = new();` — Creates an empty `LoginDto` object to hold the form data. Initialized with `new()` to avoid null reference errors.
- ▶ `Http.PostAsJsonAsync("api/auth/login", LoginRequest)` — Sends a POST request to the API with the `LoginDto` as JSON body. Returns an `HttpResponseMessage`.
- ▶ `response.IsSuccessStatusCode` — Returns true if the HTTP status code is in the 200-299 range (e.g., 200 OK).
- ▶ `response.Content.ReadFromJsonAsync<AuthResponse>()` — Reads the JSON body of the response and deserializes it into an `AuthResponse` object (which contains the JWT token).
- ▶ `JS.InvokeVoidAsync("localStorage.setItem", "token", authResponse!.AccessToken)` — Calls browser's JavaScript `localStorage.setItem("token", "...")` to save the JWT token. The `!` asserts `authResponse` is not null.
- ▶ `AuthServiceProvider.NotifyUserLoggedIn(authResponse.AccessToken)` — Tells Blazor's auth system "a user just logged in" with this token, causing the UI to immediately update (`NavMenu` changes from Login/Register to Logout).
- ▶ `Nav.NavigateTo("/")` — Redirects the user to the home page after successful login.

④ Step 4: Create Register Page

Purpose: Build a registration form that collects Email, Password, ConfirmPassword and Role, then calls the API to create a new user.

File: EmployeeClient/Pages/Register.razor

```
@page "/register"
@inject HttpClient Http

<h3 class="text-primary">Register</h3>

<div class="col-4">
    <EditForm Model="RegisterRequest" OnValidSubmit="HandleRegister">
        <div class="mb-2">
            <label class="fw-bold">Email</label>
            <InputText class="form-control" @bind-Value="RegisterRequest.Email"/>
            <ValidationMessage For="()"=>RegisterRequest.Email"/>
        </div>
        <div class="mb-2">
            <label class="fw-bold">Password</label>
            <InputText type="password" class="form-control" @bind-Value="RegisterRequest.Password"/>
            <ValidationMessage For="()"=>RegisterRequest.Password"/>
        </div>
        <div class="mb-2">
            <label class="fw-bold">Confirm Password</label>
            <InputText type="password" class="form-control" @bind-Value="RegisterRequest.ConfirmPassword"/>
            <ValidationMessage For="()"=>RegisterRequest.ConfirmPassword"/>
        </div>
        <div class="mb-2">
            <label class="fw-bold">Role</label>
            <InputSelect class="form-control" @bind-Value="RegisterRequest.Role">
                <option value="User">User</option>
            </div>
        </div>
    </EditForm>
</div>
```



```

        <option value="Admin">Admin</option>
    </InputSelect>
</div>
<button class="btn btn-primary">Register</button>
</EditForm>
</div>

@code {
    private RegisterDto RegisterRequest = new();

    private async Task HandleRegister()
    {
        var response = await Http.PostAsJsonAsync("api/auth/register", RegisterRequest);
        if (response.IsSuccessStatusCode)
            Console.WriteLine("User Registered Successfully");
        else
            Console.WriteLine("Registration Failed");
    }
}

```

Key Points (Differences from Login):

- ▶ `<InputSelect @bind-Value="RegisterRequest.Role">` — Blazor's dropdown component for use inside EditForm. Always use InputSelect (not plain HTML `<select>`) inside an EditForm so data binding works correctly.
- ▶ `<option value="User">User</option>` — Each option has a value that gets bound to `RegisterRequest.Role` when selected.
- ▶ `RegisterDto RegisterRequest = new()` — `RegisterDto.Role` has a default value of "User" in the Shared project, so the dropdown starts pre-selected on "User".

⑤ Step 5: Store JWT Token after Login

Purpose: After a successful login, save the JWT token to the browser's localStorage so it persists across page navigations and can be attached to future API calls.

What is a JWT Token?

A JWT (JSON Web Token) is a string with 3 parts separated by dots: `Header.Payload.Signature`

The **Payload** (middle part) contains **claims** — facts about the user encoded in Base64:

Claim	Value Example	Meaning
sub	"a1b2c3-guid"	User's unique ID
email	"john@test.com"	User's email
jti	"some-guid"	Unique token ID
nameidentifier	"a1b2c3-guid"	User's ID (again)
role	"Admin"	User's role

What is localStorage?

Think of localStorage as a **small notebook the browser keeps for your app**. You can save key-value pairs that survive page refreshes. In Blazor WASM, we use `IJSRuntime` to call JavaScript's `localStorage` API.

Key Code in `HandleLogin`:

```
// Read the AuthResponse JSON from the API response
var authResponse = await response.Content.ReadFromJsonAsync<AuthResponse>();

// Save the JWT token to browser LocalStorage under the key "token"
await JS.InvokeVoidAsync("localStorage.setItem", "token", authResponse!.AccessToken);

// Tell Blazor the user is now logged in
AuthStateProvider.NotifyUserLoggedIn(authResponse.AccessToken);
```

⑥ Step 6: Create CustomAuthStateProvider

Purpose: Blazor needs to know "is the user logged in or not?" at all times. The AuthenticationStateProvider is the class that answers this question by reading and parsing the JWT token.

File: EmployeeClient/Auth/CustomAuthStateProvider.cs

```
using Microsoft.AspNetCore.Components.Authorization;
using Microsoft.JSInterop;
using System.Security.Claims;
using System.Text.Json;

namespace EmployeeClient.Auth
{
    public class CustomAuthStateProvider : AuthenticationStateProvider
    {
        private readonly IJSRuntime _js;
        private ClaimsPrincipal _currentUser = new ClaimsPrincipal(new ClaimsIdentity());

        public CustomAuthStateProvider(IJSRuntime js)
        {
            _js = js;
        }
    }
}
```

```

}

// Blazor calls this automatically to get the current auth state
public override async Task<AuthenticationState> GetAuthenticationStateAsync()
{
    // If user is already cached in memory, return immediately (no JS call needed)
    if (_currentUser.Identity?.IsAuthenticated == true)
        return new AuthenticationState(_currentUser);

    // Read the token from browser LocalStorage
    var token = await _js.InvokeAsync<string>("localStorage.getItem", "token");

    // No token = anonymous (not logged in)
    if (string.IsNullOrEmpty(token))
        return new AuthenticationState(new ClaimsPrincipal(new ClaimsIdentity()));

    // Parse the JWT token to extract user claims
    var claims = ParseClaimsFromJwt(token);
    var identity = new ClaimsIdentity(claims, "jwt");
    _currentUser = new ClaimsPrincipal(identity);
    return new AuthenticationState(_currentUser);
}

// Call this after Login – updates Blazor's auth state immediately
public void NotifyUserLoggedIn(string token)
{
    var claims = ParseClaimsFromJwt(token);
    var identity = new ClaimsIdentity(claims, "jwt");
    _currentUser = new ClaimsPrincipal(identity);
    NotifyAuthenticationStateChanged(Task.FromResult(new AuthenticationState(_currentUser)));
}

// Call this after Logout – sets user back to anonymous
public void NotifyUserLoggedOut()
{
    _currentUser = new ClaimsPrincipal(new ClaimsIdentity());
    NotifyAuthenticationStateChanged(Task.FromResult(new AuthenticationState(_currentUser)));
}

// Decodes the JWT payload and extracts claims
private IEnumerable<Claim> ParseClaimsFromJwt(string jwt)
{

```

```

var claims = new List<Claim>();

// JWT = Header.Payload.Signature - take the middle part
var payload = jwt.Split(".")[1];

// Base64 strings must have length divisible by 4. JWT removes padding, so we add it back
switch (payload.Length % 4)
{
    case 2: payload += "=="; break;
    case 3: payload += "="; break;
}

// Decode the Base64 string to raw bytes (which represent JSON)
var jsonBytes = Convert.FromBase64String(payload);

// Deserialize the bytes into a dictionary of key-value pairs
var keyValuePairs = JsonSerializer.Deserialize<Dictionary<string, JsonElement>>(jsonBytes);

if (keyValuePairs == null) return claims;

// Convert each key-value pair into a Claim object
foreach (var kvp in keyValuePairs)
{
    // Some claims (like role) can be an array, handle both cases
    if (kvp.Value.ValueKind == JsonValueKind.Array)
    {
        foreach (var element in kvp.Value.EnumerateArray())
            claims.Add(new Claim(kvp.Key, element.ToString()));
    }
    else
    {
        claims.Add(new Claim(kvp.Key, kvp.Value.ToString()));
    }
}
return claims;
}
}
}

```

Key Concept Explanations:

ClaimsIdentity vs ClaimsPrincipal vs AuthenticationState

Class	Analogy	Role
ClaimsIdentity	The ID Card	Holds the user's claims (email, role) and how they were authenticated
ClaimsPrincipal	The Person holding the ID card	Wraps one or more ClaimsIdentity objects
AuthenticationState	The Security Checkpoint result	Wraps the ClaimsPrincipal — this is what Blazor uses

⚠ **Critical:** `new ClaimsIdentity(claims, "jwt")` — The second argument "jwt" marks the user as **authenticated**. Without it (`new ClaimsIdentity(claims)`), even with claims, the user is treated as **anonymous**!

💡 **Base64 Padding:** JWT tokens intentionally strip the = padding characters from the Base64-encoded payload to keep URLs short. We add them back before decoding:

- ▶ `length % 4 == 2` → add ==
- ▶ `length % 4 == 3` → add =
- ▶ `length % 4 == 0` → no padding needed

💡 **NotifyAuthenticationStateChanged:** This is a built-in method from the base class `AuthenticationStateProvider`. Calling it immediately pushes the new auth state to all `AuthorizeView` and `CascadingAuthenticationState` components — no page refresh needed!

⑦ Step 7: Register Services in Program.cs

Purpose: Register the auth services in the DI container so Blazor knows to use our custom provider for authentication.

File: EmployeeClient/Program.cs (complete)

```
using EmployeeClient;
using EmployeeClient.Auth;
using Microsoft.AspNetCore.Components.Authorization;
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;

var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

// Configure HttpClient to point to the API
builder.Services.AddScoped(sp => new HttpClient
{
    BaseAddress = new Uri("https://localhost:7021/")
});

// Enable Blazor's authorization system
builder.Services.AddAuthorizationCore();

// Register CustomAuthstateProvider so it can be injected by its own type
builder.Services.AddScoped<CustomAuthstateProvider>();

// Tell Blazor to use our CustomAuthstateProvider when it needs AuthenticationstateProvider
// Using GetRequiredService ensures BOTH registrations use the SAME instance
builder.Services.AddScoped<AuthenticationstateProvider>(sp =>
    sp.GetRequiredService<CustomAuthstateProvider>());

await builder.Build().RunAsync();
```

Why Two Registrations?

💡 We need two registrations:

1. `AddScoped<CustomAuthStateProvider>()` — so that `Login.razor` and `MainLayout.razor` can inject and call `NotifyUserLoggedIn()` / `NotifyUserLoggedOut()`
2. `AddScoped<AuthenticationStateProvider>(...)` — so that Blazor's built-in `CascadingAuthenticationState` uses our custom provider

⚠ **Important:** Using `sp.GetRequiredService<CustomAuthStateProvider>()` ensures BOTH registrations resolve to the **same object instance**. If you write `AddScoped<AuthenticationStateProvider, CustomAuthStateProvider>()` instead, two separate instances are created — notifications (login/logout) won't reach the UI!

⑧ Step 8: Update App.razor

Purpose: Wrap the entire app with Blazor's authentication infrastructure so every page knows the current auth state and protected routes can be enforced.

File: EmployeeClient/App.razor

```
<CascadingAuthenticationState>
  <Router AppAssembly="@typeof(App).Assembly" NotFoundPage="typeof(Pages.NotFound)">
    <Found Context="routeData">
      <AuthorizeRouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)">
        <NotAuthorized>
          <p>You are not authorized. Please <a href="/login">login</a>.</p>
        </NotAuthorized>
      </AuthorizeRouteView>
      <FocusOnNavigate RouteData="@routeData" Selector="h1"/>
    </Found>
  </Router>
</CascadingAuthenticationState>
```



```
        </Found>
    </Router>
</CascadingAuthenticationState>
```

Line-by-Line Explanation:

- ▶ `<CascadingAuthenticationState>` — Makes the authentication state available to ALL child components in the tree via Blazor's cascading value mechanism. Every `AuthorizeView` inside can access the current user's auth state.
- ▶ `<AuthorizeRouteView>` (replaces the default `<RouteView>`) — Checks if the target page requires authorization (via `@attribute [Authorize]`) before rendering it. If the user doesn't have access, shows `<NotAuthorized>` content instead.
- ▶ `<NotAuthorized>` — Content shown when a user tries to access a page they're not authorized for. We show a friendly message with a link to the login page.

Component	Before	After
Wrapper	(none)	CascadingAuthenticationState
Route renderer	RouteView	AuthorizeRouteView

⑨ Step 9: Update MainLayout.razor (Navigation + Auth)

Purpose: Make the navigation bar show different links depending on whether the user is logged in, and what role they have.

File: EmployeeClient/Layout/MainLayout.razor

```

@inherits LayoutComponentBase
@Inject IJSRuntime JS
@Inject CustomAuthProvider AuthServiceProvider
@Inject NavigationManager Nav

<div>
    <nav class="navbar navbar-expand navbar-dark bg-dark">
        <a class="navbar-brand" href="">Employee App</a>
        <ul class="navbar-nav">

            <AuthorizeView>                                <!-- checks: is any user logged in? -->
                <Authorized>
                    <!-- SHOWN TO ALL LOGGED IN USERS -->
                    <li class="nav-item">
                        <NavLink class="nav-link" href="employees">Employee List</NavLink>
                    </li>

                    <!-- NESTED: shown only to Admin -->
                    <AuthorizeView Roles="Admin">
                        <Authorized Context="adminContext">
                            <li class="nav-item">
                                <NavLink class="nav-link" href="add">Add Employee</NavLink>
                            </li>
                        </Authorized>
                    </AuthorizeView>

                    <!-- LOGOUT: shown to all logged in users -->
                    <li class="nav-item">
                        <button class="btn btn-link nav-link" @onclick="HandleLogout">Logout</button>
                    </li>
                </Authorized>

                <NotAuthorized>
                    <!-- SHOWN WHEN NOT LOGGED IN -->
                    <li class="nav-item">
                        <NavLink class="nav-link" href="login">Login</NavLink>
                    </li>
                    <li class="nav-item">
                        <NavLink class="nav-link" href="register">Register</NavLink>
                    </li>
                </NotAuthorized>
            </ul>
        </nav>
    </div>

```

```

        </AuthorizeView>

    </ul>
</nav>

<main>
    <article class="content px-4">
        @Body
    </article>
</main>
</div>

@code {
    private async Task HandleLogout()
    {
        // Remove the token from browser LocalStorage
        await JS.InvokeVoidAsync("localStorage.removeItem", "token");
        // Tell Blazor the user has logged out
        AuthStateProvider.NotifyUserLoggedOut();
        // Redirect to the login page
        Nav.NavigateTo("login");
    }
}

```

AuthorizeView Explained:

Usage	Shows Authorized content when
<AuthorizeView>	User is logged in (any role)
<AuthorizeView Roles="Admin">	User is logged in AND has "Admin" role
<AuthorizeView Roles="Admin,User">	User is logged in AND has "Admin" OR "User" role

⚠ **Nested AuthorizeView Context:** When nesting two AuthorizeView components, the inner <Authorized> block MUST have a different Context name to avoid ambiguity:

```
<Authorized Context="adminContext">
```

💡 **NavMenu Rules Summary:**

- Not logged in → Login + Register visible
- Logged in as User → Employee List + Logout visible
- Logged in as Admin → Employee List + Add Employee + Logout visible

⑩ Step 10: Send Token with API Requests

Purpose: Protected API endpoints require a JWT token in the request's Authorization header. We must attach the token saved in localStorage to every API call.

What is the Authorization Header?

Every HTTP request to a protected endpoint must include:

Authorization: Bearer eyJhbGci... (the full JWT token)

The API reads this header, validates the token, and grants or denies access.

In Employees.razor (Manual approach):

```

@Inject IJSRuntime JS

@code {
    protected override async Task OnInitializedAsync()
    {
        // Step 1: Read token from LocalStorage
        var token = await JS.InvokeAsync<string>("localStorage.getItem", "token");

        // Step 2: Attach token to ALL future requests from this HttpClient instance
        Http.DefaultRequestHeaders.Authorization =
            new System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", token);

        // Step 3: Call the API – token is now included automatically
        EmployeeList = await Http.GetFromJsonAsync<List<EmployeeDto>>("api/employees") ?? new();
    }
}

```

Line-by-Line Explanation:

- ▶ `JS.InvokeAsync<string>("localStorage.getItem", "token")` — Calls JavaScript's `localStorage.getItem("token")` from C#. Returns the JWT string we saved during login.
- ▶ `new AuthenticationHeaderValue("Bearer", token)` — Creates the header value `Bearer eyJhbGciOi...` "Bearer" is the standard scheme for JWT tokens.
- ▶ `Http.DefaultRequestHeaders.Authorization = ...` — Sets the Authorization header on the HttpClient. Once set, ALL subsequent requests from this instance will include this header automatically.

 **Note:** This manual approach must be repeated in every component that calls a protected API endpoint. An alternative is to use a **DelegatingHandler** which automatically attaches the token to every request without repeating the code.

⑪ Step 11: Role-Based Buttons in Employee List

Purpose: Show Edit and Delete buttons in the employee table, but only to users who have the permission to perform those actions based on their role.

API Authorization Rules (Reminder):

Action	Who Can Do It
Edit (PUT)	Admin + User
Delete (DELETE)	Admin only

File: EmployeeClient/Pages/Employees.razor (Actions Column)

```
<!-- In <thead> -->
<th>Actions</th>

<!-- In <tbody> @foreach Loop -->
<td>
    <!-- Edit button: visible to Admin AND User -->
    <AuthorizeView Roles="Admin,User">
        <Authorized Context="editCtx">
            <NavLink class="btn btn-sm btn-warning"
                href="@($"edit/{emp.Id}")">Edit</NavLink>
        </Authorized>
    </AuthorizeView>

    <!-- Delete button: visible to Admin ONLY -->
    <AuthorizeView Roles="Admin">
        <Authorized Context="deleteCtx">
            <button class="btn btn-sm btn-danger"
```

```

                @onclick="()=>HandleDelete(emp.Id)">Delete</button>
            </Authorized>
        </AuthorizeView>
    </td>

@code {
    private async Task HandleDelete(int id)
    {
        // Read token and attach to header
        var token = await JS.InvokeAsync<string>("localStorage.getItem", "token");
        Http.DefaultRequestHeaders.Authorization =
            new System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", token);

        // Call DELETE endpoint
        var response = await Http.DeleteAsync($"api/employees/{id}");

        if (response.IsSuccessStatusCode)
        {
            // Remove from local list to update UI without re-fetching
            EmployeeList?.RemoveAll(e => e.Id == id);
        }
    }
}

```

Line-by-Line Explanation:

- ▶ `<AuthorizeView Roles="Admin,User">` — Shows Edit button only when the logged-in user's role is either "Admin" OR "User".
- ▶ `<AuthorizeView Roles="Admin">` — Shows Delete button only when the logged-in user's role is "Admin".
- ▶ `Context="editCtx"` and `Context="deleteCtx"` — Each nested AuthorizeView needs a unique Context name to avoid naming conflicts.
- ▶ `href="@($"edit/{emp.Id}")"` — Dynamic route using C# string interpolation. Navigates to /edit/5 (where 5 is the employee ID).
- ▶ `@onclick="()=>HandleDelete(emp.Id)"` — Lambda expression that passes the current employee's ID to the delete method. Without the lambda, all delete buttons would share the same ID.

- ▶ `Http.DeleteAsync($"api/employees/{id}")` — Sends an HTTP DELETE request to the API endpoint for the specific employee.
- ▶ `EmployeeList?.RemoveAll(e => e.Id == id)` — Removes the deleted employee from the in-memory list to update the UI instantly without making another GET request.

✓ Summary — What We Built

Step	What We Did	Files Changed
1	Configure HttpClient to point to the API	Program.cs
2	Add global @using directives	_Imports.razor
3	Create Login form page	Pages/Login.razor
4	Create Register form page	Pages/Register.razor
5	Save JWT token to localStorage after login	Pages/Login.razor
6	Create custom AuthenticationStateProvider	Auth/CustomAuthProvider.cs
7	Register auth services correctly	Program.cs
8	Wrap app with auth infrastructure	App.razor
9	Role-based NavMenu + Logout	Layout/MainLayout.razor
10	Attach JWT token to protected API calls	Pages/Employees.razor

11	Show Edit/Delete based on user role	Pages/Employees.razor
----	-------------------------------------	-----------------------

Final NavBar Behavior:

User State	Visible Nav Items
Not logged in	Login, Register
Logged in as User	Employee List, Logout
Logged in as Admin	Employee List, Add Employee, Logout

Key Concepts Learned:

-  **JWT Authentication** — Token-based auth where the server issues a signed token the client stores and re-sends
-  **localStorage** — Browser storage for persisting the token across page navigations
-  **AuthenticationStateProvider** — The bridge between the JWT token and Blazor's auth system
-  **CascadingAuthenticationState** — Makes auth state available to the entire component tree
-  **AuthorizeView** — Conditionally shows/hides UI elements based on auth state and roles
-  **DI Instance Sharing** — Using `GetRequiredService` to ensure both DI registrations share the same object